

LRN Analytics API

Read-only access to LRN reference data: CPT and panel reimbursement averages with mode and median rates, the global payer policy catalogue, and lab-level payer records with their mapping to it.

BASE URL `https://<host>/LRNapi`

Confirm the exact host with your LRN contact. All paths below are relative to it.

00 Getting started

Every data endpoint is a `GET` returning JSON, and every request needs a bearer token. Three steps to your first response.

1 Receive your credentials

LRN issues a **client id** and **client secret**. The secret is shown once and cannot be recovered — store it in your secret manager, never in source control.

2 Exchange them for a token

Call `POST /api/auth/token`. You get a JWT valid for one hour. The secret itself is never sent to a data endpoint.

3 Call the data endpoints

Send `Authorization: Bearer <token>` on every request. Cache the token and reuse it until it is near expiry.

01 Authentication

A standard client-credentials exchange. Your credentials identify a machine client, not a person — there is no interactive login step.

POST `/api/auth/token`

Exchange client credentials for a bearer token.

REQUEST

```
POST /api/auth/token
Content-Type: application/json

{
  "clientId": "your-client-id",
  "clientSecret": "your-client-secret"
}
```

`grantType` may be included as `"client_credentials"`; it is the only grant supported and may be omitted.

200 OK

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "expires_at_utc": "2026-08-14T16:05:36.999Z",
  "roles": ["ETL"]
}
```

ERROR RESPONSES

STATUS	MEANING
400	<code>clientId</code> or <code>clientSecret</code> missing, or an unsupported <code>grantType</code>
401	Client id or secret is not valid, or the client is disabled
429	Too many failed attempts. The client id is locked out for a short period
500	Server-side signing configuration problem — contact LRN support

Using and refreshing the token

Send it as a bearer token on every data request. A missing, expired or invalid token returns `401` with a JSON body explaining which.

```
Authorization: Bearer <access_token>
```

TOKEN LIFECYCLE

`expires_in` is seconds — one hour by default. **Cache the token and reuse it**; do not request a new one per call. Re-request when it has under about five minutes left, or when a call returns `401`. Repeated failed token requests will lock the client id out temporarily.

Worked example

CURL

```
TOKEN=$(curl -s -X POST https://<host>/LRNApi/api/auth/token \
  -H "Content-Type: application/json" \
  -d '{"clientId":"your-client-id","clientSecret":"your-client-secret"}' \
  | jq -r .access_token)

curl -s "https://<host>/LRNApi/api/analytics/cpt-lookup?windowType=YTD&pageSize=100" \
  -H "Authorization: Bearer $TOKEN"
```

PYTHON

```

import requests

BASE = "https://<host>/LRNApi"

token = requests.post(f"{BASE}/api/auth/token", json={
    "clientId": "your-client-id",
    "clientSecret": "your-client-secret",
}, timeout=30).json()["access_token"]

headers = {"Authorization": f"Bearer {token}"}

page = 1
# Page sequentially off totalPages, not a fixed count.
while True:
    r = requests.get(f"{BASE}/api/analytics/cpt-lookup",
                     params={"windowType": "YTD", "page": page, "pageSize": 500},
                     headers=headers, timeout=120)
    r.raise_for_status()
    body = r.json()
    for row in body["items"]:
        ... # handle row
    if page >= body["totalPages"]:
        break
    page += 1

```

C#

```

using var http = new HttpClient { BaseAddress = new Uri("https://<host>/LRNApi/") };

var auth = await http.PostAsJsonAsync("api/auth/token", new
{
    clientId = "your-client-id",
    clientSecret = "your-client-secret"
});
var token = (await auth.Content.ReadFromJsonAsync<JsonElement>())
    .GetProperty("access_token").GetString();

http.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);
var rows = await http.GetFromJsonAsync<JsonElement>(
    "api/analytics/cpt-lookup?windowType=YTD&pageSize=500");

```

02 Conventions

These hold across every endpoint in this guide.

Paging

List endpoints take `page` (1-based) and `pageSize`, and return an envelope:

```
{ "items": [ ... ], "page": 1, "pageSize": 50, "totalCount": 1234, "totalPages": 25 }
```

`pageSize` is clamped to **10–1000** on every list endpoint — a larger value is silently reduced and a smaller one silently raised to 10. Always page off `totalPages` rather than assuming a fixed count.

Sorting and filtering

`sortColumn` pairs with `sortDirection` (`asc` | `desc`). Only the columns listed per endpoint are accepted; anything else falls back to that endpoint's default order. All filter parameters are optional and combine with AND — omitted or empty means no restriction, so leaving out `labId` returns every lab.

Nulls

READ THIS BEFORE MODELLING THE DATA

A `null` numeric means the value **does not exist** for that combination — not zero. Mode and median rates come from different source tables than the averages, and the payer names only partly overlap between them, so a missing rate on a row that has averages is normal rather than an error.

Errors

STATUS	MEANING
401	Unauthenticated — missing, expired or invalid token
403	Your token's role cannot view this resource
404	Unknown id
500	Server error. The response carries an <code>X-Correlation-ID</code> header — quote it to support

Dates are ISO-8601 UTC.

CPT & Panel Lookup

Reimbursement averages joined to mode and median rates. Windows are `YTD`, `Rolling180` and `Rolling90`; omit `windowType` to get all three.

GET `/api/analytics/cpt-lookup`

CPT-level rows, paged, with a summary computed over the whole filtered set.

PARAMETERS

PARAMETER	TYPE	NOTES
<code>cptCode</code>	string	Exact CPT code
<code>panelName</code>	string	
<code>payer</code>	string	Matches the payer display name
<code>windowType</code>	string	<code>YTD</code> <code>Rolling180</code> <code>Rolling90</code>
<code>labId</code>	int	Omit for all labs
<code>sortColumn</code>	string	See below
<code>sortDirection</code>	string	<code>asc</code> <code>desc</code>
<code>page, pageSize</code>	int	Default <code>1</code> / <code>50</code> , max <code>1000</code>

SORTABLE COLUMNS

`cptCode` , `panelName` , `payerDisplayName` , `windowType` , `labName` , `avgUnits` , `avgChargeAmountPerUnit` , `avgAllowedAmountPerUnit` , `avgPaidAmountPerUnit` , `medianPaidAmount` , `paidLineCount` , `totalLineCount` , `deniedLineCount` , `modeAllowedAmount` , `modeInsurancePaymentAmount` , `allowedAmountPerUnitMode` , `insurancePaymentPerUnitMode` , `medianAllowedAmount` , `medianInsurancePaymentAmount` , `allowedAmountPerUnitMedian` , `insurancePaymentPerUnitMedian` , `denialRate` .

RESPONSE FIELDS

FIELD	TYPE	NOTES
labId, labName	int, string	
cptCode, panelName	string	
payerDisplayName, payerCommonCode, globalPayerId	string, string, int	
windowType, startDate, endDate, asOfDate	string, date	Window the figures cover
avgUnits	int	
avgChargeAmountPerUnit, avgAllowedAmountPerUnit, avgPaidAmountPerUnit, avgPatientResponsibilityPerUnit	decimal	Per unit
medianPaidAmount, p25PaidAmount, p75PaidAmount	decimal	Paid distribution
paidLineCount, totalLineCount, deniedLineCount, adjustedLineCount	int	
denialRate	decimal	Denied ÷ total as a percentage, 1 dp; null when there are no lines
modeAllowedAmount, modeInsurancePaymentAmount, allowedAmountPerUnitMode, insurancePaymentPerUnitMode	decimal	From LabModes
medianAllowedAmount, medianInsurancePaymentAmount, allowedAmountPerUnitMedian, insurancePaymentPerUnitMedian	decimal	From LabMedians
modeMatch, medianMatch	string	How the rate was resolved. "payer" = this payer's own rate; "lab" = the lab- wide fallback because no row exists for this payer; null = no rate at all

INTERPRETING MODEMATCH AND MEDIANMATCH

Treat "lab" as materially less specific than "payer". A lab-wide fallback rate is an average across every payer at that lab, so it should not be presented as that payer's negotiated rate.

The `summary` block carries `rowCount`, `avgAllowed`, `avgPaid` and `denialRate` computed over **every** row matching the filters, not just the current page.

GET `/api/analytics/panel-lookup`

Panel-level rows. Same parameters as the CPT endpoint, minus `cptCode`.

SORTABLE COLUMNS

`panelName`, `payerId`, `payerDisplayName`, `windowType`, `labName`, `avgChargeAmount`, `avgAllowedAmount`, `avgPaidAmount`, `medianPaidAmount`, `p25PaidAmount`, `p75PaidAmount`, `paidLineCount`, `totalLineCount`, `deniedLineCount`, `modeAllowedAmount`, `modeInsurancePaymentAmount`, `modeCptCount`, `denialRate`.

RESPONSE FIELDS

Items carry `labId`, `labName`, `panelName`, `payerId`, `payerDisplayName`, the window fields, `avgChargeAmount`, `avgAllowedAmount`, `avgPaidAmount`, `avgPatientResponsibility`, `medianPaidAmount`, `p25PaidAmount`, `p75PaidAmount`, the four line counts, `denialRate`, `modeAllowedAmount`, `modeInsurancePaymentAmount`, `modeCptCount` and `modeMatch`.

PANEL MODE FIGURES ARE DERIVED

`LabModes` is CPT-level, so panel mode figures are the **average of the per-CPT modes** across the panel. `modeCptCount` says how many CPTs that average covers — a value of 1 is not a broad panel rate.

GET `/api/analytics/cpt-lookup/windows`

Also: `/api/analytics/panel-lookup/windows`

Every window (`YTD`, `Rolling180`, `Rolling90`) for one combination, for side-by-side comparison. Takes the same filters and returns a plain array of the row shape above — **not** the paged envelope.

GET `/api/analytics/cpt-lookup/options`

Also: `/api/analytics/panel-lookup/options`

Autocomplete values. `field` is `cptCode` | `panelName` | `payer` (`panelName` | `payer` on the panel endpoint), plus optional `term` and `labId`. Returns a JSON array of strings.

GET /api/analytics/lookup-labs

Labs that have averages data.

```
[{ "labId": 4, "labName": "Cove" }, ...]
```

LAB NAMES ARE NOT UNIQUE

Two different `labId` values can both be named "NorthWest". Always key on `labId` and treat the name as a display label only.

GET /api/analytics/lab-modes

Also: `/api/analytics/lab-medians`, and `/api/analytics/labs` for the labs present in these two tables.

The rate tables exposed directly. Filters: `payerName`, `panelName`, `cptCode`, `labId`, `sortColumn`, `sortDirection`, `page`, `pageSize` (default 25).

Rows carry `payerName`, `panelName`, `cptCode`, `allowedAmount`, `insurancePayment`, `distinctAllowedPaymentCount`, `labName`, plus `modeAllowedAmount`, `modeInsurancePaymentAmount`, `allowedAmountPerUnitMode`, `insurancePaymentPerUnitMode` for modes, or the `median*` equivalents for medians.

04 **Payer Policy Insurance Master**

The global payer catalogue — one record per payer policy, independent of any lab.

GET `/api/master-values/payer-policy-insurance`

Paged list.

PARAMETERS

PARAMETER	TYPE	NOTES
<code>search</code>	string	Free text across the main name and code columns
<code>globalPayerCode</code> , <code>payerName</code> , <code>payerShortCode</code> , <code>planType</code> , <code>payerState</code>	string	
<code>globalPayerId</code>	int	
<code>isActive</code>	string	
<code>sortColumn</code> , <code>sortDirection</code>	string	See below
<code>page</code> , <code>pageSize</code>	int	Default 1 / 25

SORTABLE COLUMNS

`globalPayerId` , `globalPayerCode` , `payerGroupCode` , `benefitAdminCode` , `benefitAdministrator` , `payerNameRaw` (Or `payerName`) , `payerNameNormalized` , `payerShortCode` , `planType` , `payerState` , `isActive` , `remarks` , `payerFamily` , `payerFamilySource` . Default order is `ppInsuranceMasterId` descending.

RESPONSE FIELDS

FIELD	TYPE
<code>ppInsuranceMasterId</code>	int — primary key
<code>globalPayerId</code>	int
<code>globalPayerCode</code>	string — always present
<code>payerGroupCode</code>	int
<code>benefitAdminCode</code> , <code>benefitAdministrator</code>	string
<code>payerNameRaw</code> , <code>payerNameNormalized</code> , <code>payerShortCode</code>	string
<code>planType</code> , <code>payerState</code> , <code>isActive</code> , <code>remarks</code>	string
<code>payerFamily</code> , <code>payerFamilySource</code>	string — brand-family classification

GET

/api/master-values/payer-policy-insurance/{id}

One record by `ppInsuranceMasterId` . Returns `404` when it does not exist.

Insurance Payer Master

Lab-level payer records and how each maps to the global catalogue.

GET `/api/master-values/insurance-payers`

Paged list.

PARAMETERS

PARAMETER	TYPE	NOTES
<code>search</code>	string	Free text
<code>labId</code>	int	
<code>payerCode, payerName</code>	string	
<code>globalPayerId</code>	int	
<code>isActive</code>	string	
<code>mappingStatus</code>	string	<code>Mapped</code> <code>Unmapped</code> <code>Unmapped - Pending Review</code> <code>No Match Found</code>
<code>sortColumn, sortDirection</code>	string	See below
<code>page, pageSize</code>	int	Default <code>1 / 25</code>

SORTABLE COLUMNS

`payerCode` , `payerNameRaw` , `payerNameNormalized` , `globalPayerID` , `payerGroupCode` , `payerCommonCode` , `parent` , `planType` , `mcoType` , `payerState` , `isActive` , `benefitAdminCode` , `benefitAdministrator` , `labName` , `labState` , `labStateCode` , `remarks` , `mappingStatus` , `mappedSource` , `mappedBy` , `mappedOn` .
Default order is `labInsuranceMasterId` descending.

RESPONSE FIELDS

FIELD	TYPE	NOTES
labInsuranceMasterId	int	Primary key
payerCode, payerNameRaw, payerNameNormalized	string	
globalPayerID	int	Link to the policy master. null means unmapped
payerGroupCode, payerCommonCode, parent	string	
planType, mcoType, payerState, isActive	string	
benefitAdminCode, benefitAdministrator, remarks	string	
labName, labId, labState, labStateCode	string, int	
mappingStatus, mappedBy, mappedSource, mappedOn	string, date	mappedSource is System or User

JOINING THE TWO MASTERS

Join `globalPayerID` here to `globalPayerId` on the payer policy master to resolve a lab payer to its global record.

GET `/api/master-values/insurance-payers/{id}`

One record by `labInsuranceMasterId`. Returns `404` when it does not exist.

GET `/api/master-values/insurance-payers/labs`

Labs present in the payer master.

Practical notes

Access is read-only

Standard external credentials are issued with a view-only role. `POST`, `PUT` and import endpoints exist on these routes for internal use and will return `403`.

Lab scope

PASS LABID EXPLICITLY

Your token may name specific labs, but these endpoints do not currently filter by that — omitting `labId` returns rows for every lab. Pass it explicitly if you only want your own.

Volume and timeouts

- An unfiltered CPT lookup spans every lab, payer and window and is large. Filter by `labId` and `windowType` where you can.
- Use `pageSize` of 500–1000 and page **sequentially**, not in parallel.
- Broad queries can take tens of seconds. Set a client timeout of at least 120 seconds.
- Retry `500` and `503` with exponential backoff. Do not retry `400`, `401` or `403`.

Schema stability

Fields may be **added** to responses without notice — deserialize tolerantly and ignore unknown properties. Removals and renames will be announced in advance.

Support

Quote the `X-Correlation-ID` response header, the endpoint, the exact query string and the UTC timestamp.

Endpoint summary

METHOD	PATH	RETURNS
POST	/api/auth/token	Bearer token
GET	/api/analytics/cpt-lookup	Paged CPT rows + summary
GET	/api/analytics/cpt-lookup/windows	All windows for one combination
GET	/api/analytics/cpt-lookup/options	Autocomplete values
GET	/api/analytics/panel-lookup	Paged panel rows + summary
GET	/api/analytics/panel-lookup/windows	All windows for one combination
GET	/api/analytics/panel-lookup/options	Autocomplete values
GET	/api/analytics/lookup-labs	Labs with averages data
GET	/api/analytics/lab-modes	Paged mode rates
GET	/api/analytics/lab-medians	Paged median rates
GET	/api/analytics/labs	Labs in the rate tables
GET	/api/master-values/payer-policy-insurance	Paged policy payers
GET	/api/master-values/payer-policy-insurance/{id}	One policy payer
GET	/api/master-values/insurance-payers	Paged lab payers
GET	/api/master-values/insurance-payers/{id}	One lab payer
GET	/api/master-values/insurance-payers/labs	Labs in the payer master